Day 141: Training a Tiny Diffusion Model: Real Toy Image Generation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 21, Day 2 | Complete study guide

Watch: Denoising Diffusion Probabilistic Models from Scratch using PyTorch! (Papers in 100 Lines of Code) | Generative Image Diffusion from Scratch! PyTorch Deep Learning Tutorial (Luke Ditria)

Script: `day141_tiny_diffusion.py` · every number in this guide is copied by program from `day141_run_output.txt`, the real output of that script.

Day 140's reverse process worked on a ring, where the best denoiser could be written down. Today a network has to learn it, from the 60,000 MNIST training digits, and then generate new digits from pure noise. The first two drafts failed and both failures are kept in the lesson: with standard initialisation the network stopped learning within 100 steps, and a network that guessed the noise drew speckled noise instead of digits (FID 325.88). Reading the same network's output as the clean digit instead fixed it (FID 49.45 after 3,000 steps). The final model's samples were best after 10,000 steps (FID 36.856) and then got worse (63.350 at 30,000) while its test error kept falling: the loss alone would have hidden this.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

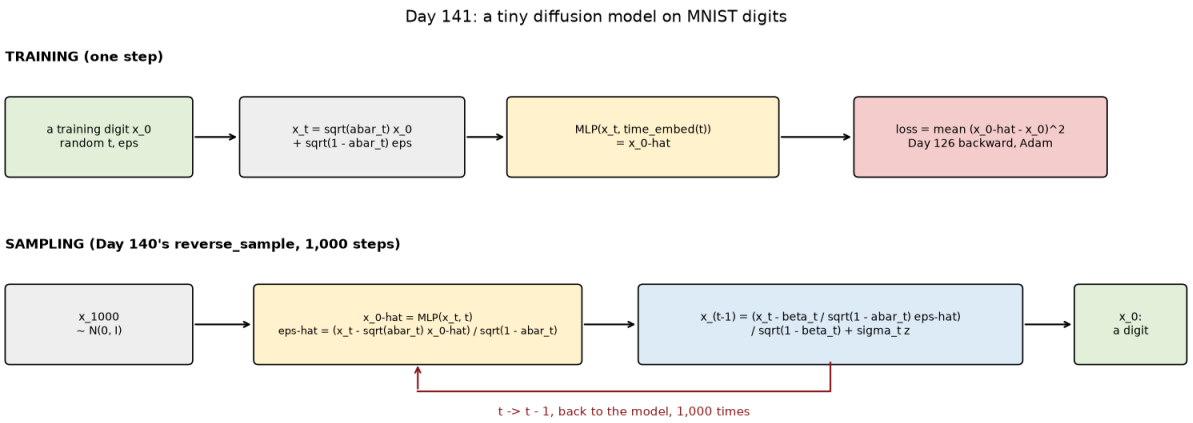
A **diffusion model** learns to undo noise. Training shows it a real image x_0 after t steps of Day 139's noising, x_t, and asks for a guess. Generation starts from pure noise and applies Day 140's reverse step 1,000 times, each time asking the network for its guess.

The guess can be stated two ways: the **noise** ε inside x_t, or the **clean image** x_0. Knowing either gives the other, so on paper they are the same model. Today shows that in practice, for a small network, the choice decides whether it works at all.

REAL-WORLD SCENARIO

A photo editor's 'remove noise' tool learns from pairs: a clean photograph and the same photograph with grain added. Shown a grainy photo, it could either paint the grain it thinks is there (and subtract it) or paint the clean photo directly. Both give the same result if done perfectly; a tool with limited capacity does much better painting the photo, because grain has no pattern to learn.

SYSTEM DIAGRAM



Training: one random digit, one random t, one squared error. Sampling: Day 140's loop, with the network's guess at every step.

Today's part for the week's build

`day141_exercise.py` is the model's parts (the straight-line guess c(t), the network's output read as x_0 or as ε, the training loop, the evaluation, the nearest-training-image check and the sample scorer), checked by hand: c(300) against a least-squares fit, the He-initialised network stalling where the zero-started one learns, and 1,000 steps of training already beating both guesses that need no training.

ACTUAL OUTPUT

```
c(300) = 0.8006; least-squares fit on 1,000 test digits: 0.7996
300 steps, output read as eps, output layer He: loss 1.016, layer-2 ReLUs on 0.01 (averages)
300 steps, output read as eps, output layer zero: loss 0.764, layer-2 ReLUs on 0.75 (averages)
1,000 steps, output read as x_0, at t = 300: x_0 error 0.0986; x_t / sqrt(abar) 1.5187; the
    average training digit 0.2689
100 samples after 1,000 steps: ink 0.163, mean distance to the nearest training digit 5.21
```

Data: Day 126's 70,000 MNIST digits, pixels scaled from [0, 1] to [–1, 1] (training images 0–59,999; test images 60,000 on, always taken in random order). The schedule, time embedding and reverse_sample are Day 140's parts; the network and Adam are Day 126's; the scorer (IS, FID, precision, recall, nearest) is Day 138's. All are imported unchanged.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: the best straight-line guess, and how ε and x_0 trade errors

DEFINITION — start here, no prior knowledge assumed

Before training anything, two facts that need only algebra. First, the best guess of the noise of the form $c(t)\cdot x_t$ has a closed form; it is a floor any network should beat. Second, since $x_t = \sqrt{A_t}\, x_0 + \sqrt{(1 - A_t)}\, \varepsilon$, a miss in one guess becomes a miss in the other, scaled by a factor that depends on t.

REAL-WORLD SCENARIO

Converting between two currencies at an exchange rate that swings wildly: a one-cent error in one currency can be a hundred-cent error in the other.

MATHEMATICS

$$c(t) = E[x_t\, \varepsilon] \,/\, E[x_t^2] = \sqrt{(1 - A_t)} \,/\, (A_t\, E[x_0^2] + 1 - A_t) \quad (x_0 \text{ and } \varepsilon \text{ independent})$$

$$\varepsilon' = (x_t - \sqrt{A_t}\, x'_0) \,/\, \sqrt{(1 - A_t)} \quad \text{so a miss } d \text{ in } x'_0 \text{ is a miss } \sqrt{(A_t \,/\, (1 - A_t))}\, d \text{ in } \varepsilon'$$

$A_t = ABAR[t]$; primes mark guesses

IMPLEMENTATION

```
def skip_coef(X2):
    m2 = float((X2.astype(np.float64) ** 2).mean())
    return np.sqrt(1 - ABAR) / (ABAR * m2 + 1 - ABAR)
```

ACTUAL OUTPUT

```
pixels scaled to [-1, 1]; E[x_0^2] over the 60,000 training digits = 0.9254 (most pixels are background, -1)
c(t) = sqrt(1 - abar_t) / (abar_t E[x_0^2] + 1 - abar_t), against c fitted by least squares on 1,000 test digits noised to
    t;
since x_t = sqrt(abar_t) x_0 + sqrt(1 - abar_t) eps, a miss of d in x_0 is a miss of sqrt(abar_t / (1 - abar_t)) d in eps
     t  sqrt(1-abar)     c(t)  fitted c  eps error of c(t) x_t  x_0 miss -> eps miss
     1        0.0100   0.0108    0.0102                1.0028               99.9950
    10        0.0435   0.0470    0.0468                0.9969               22.9513
    50        0.1702   0.1835    0.1824                0.9668                5.7880
   100        0.3209   0.3439    0.3462                0.8905                2.9513
   200        0.5839   0.6141    0.6147                0.6427                1.3903
   300        0.7769   0.8006    0.8012                0.3785                0.8104
   500        0.9599   0.9656    0.9656                0.0731                0.2920
   700        0.9965   0.9970    0.9972                0.0064                0.0838
  1000        1.0000   1.0000    1.0000                0.0000                0.0064
at large t, x_t is nearly all noise, so c(t) x_t is nearly the right noise guess; at small t the noise is a sliver of x_t
and a straight line cannot pull it out. The last column is why the choice of target matters: at t = 1 a tiny miss in x_0
is a 100-times-larger miss in eps, at t = 1,000 a large miss in x_0 barely moves eps
Actual output from this lesson's run.
```

What it means

The formula matches the fit at every t (c(300) = 0.8006, fitted 0.8012). At large t the straight line is almost perfect (error 0.0064 at t = 700) because x_t is almost pure noise. At small t it is useless (error 0.9969 at t = 10, about the same as guessing zero): the noise is a sliver of x_t and cannot be pulled out without knowing what a digit looks like.

The last column matters for everything that follows. At t = 1 a miss in x_0 becomes a miss 99.9950 times as large in ε; at t = 1,000 it is shrunk to 0.0064 of itself. A network asked for ε and one asked for x_0 are asked the same question, but they are graded very differently at each noise level.

WHY CHOOSE THIS?

A floor that costs nothing to compute, and an exact rule for comparing the two ways of stating a guess.

WHY NOT THIS? (TRADEOFFS)

A straight line knows nothing about digits; it only says how much of x_t is noise on average.

2. First drafts that failed, and what fixed them

DEFINITION — start here, no prior knowledge assumed

The network is Day 126's dense network: x_t (784 pixels) and Day 140's 32-number time embedding in, two hidden layers of 512 ReLUs, 784 numbers out. The first draft followed Ho et al. (2020): read the output as ε, start every layer with Day 126's He initialisation. It failed twice over, and the fixes are kept here as experiments, not hidden.

REAL-WORLD SCENARIO

A new machine that jams on its first run: before blaming the design, check how it was switched on.

SPECIFICATION

Draft (a)	output read as ε; He initialisation everywhere against an output layer started at zero; 400 steps each
Draft (b)	zero start; output read as ε (512 and 1,024 wide), as ε on top of c(t) x_t, or as x_0; 3,000 steps each, then 2,000 samples scored
Shared	Adam lr 0.001, batches of 128 training digits, t uniform on 1–1,000

IMPLEMENTATION

```
def guess(net, xt, t, kind="x0", C=None):
    A = d126.forward(net, np.concatenate([xt, d140.time_embed(t)], 1).astype(np.float32), ACTS)
    o = A[-1].astype(np.float64); sa, sb = np.sqrt(ABAR[t])[:, None], np.sqrt(1 - ABAR[t])[:,
        None]
    if kind == "x0":
        x0 = o; e = (xt - sa * x0) / sb
    else:
        e = o + (C[t][:, None] * xt if kind == "eps+line" else 0.0); x0 = (xt - sb * e) / sa
    return A, e, x0
```

ACTUAL OUTPUT

```
(a) the first draft: the output read as eps, Day 126's He initialisation everywhere. Training loss, and the share of ReLUs
that are on, each averaged over the last 50 steps:
  output layer   step    loss  layer 1 on  layer 2 on
            He     50   1.093        0.35        0.06
            He    100   1.001        0.34        0.00
            He    200   1.000        0.34        0.00
            He    400   1.000        0.35        0.00
          zero     50   0.967        0.46        0.41
          zero    100   0.841        0.73        0.73
          zero    200   0.706        0.84        0.83
          zero    400   0.600        0.91        0.89
before any training, the He-initialised output's spread (sd) at t = 500: 1.362; the target eps's: 1.001
A likely reason for the stall, consistent with these numbers but not proved here: the first large steps that shrink an
oversized output switch the second layer's ReLUs off; a switched-off ReLU passes no gradient, so they stay off, and the
network outputs a constant (loss near 1.0, the error of guessing eps = 0). A zero output layer starts every guess at 0.
```

What it means

With He initialisation the second hidden layer was switched off almost completely within 100 steps (share of ReLUs on: 0.06, then 0.00) and the loss sat at 1.000, exactly the error of guessing ε = 0. The untrained output had a spread of 1.362, larger than the target's 1.001. Starting the output layer at zero kept both layers working (0.91 and 0.89 on at step 400) and the loss falling (0.600). The explanation printed with the output (ReLUs switched off while shrinking an oversized output) fits these numbers but was not proved here.

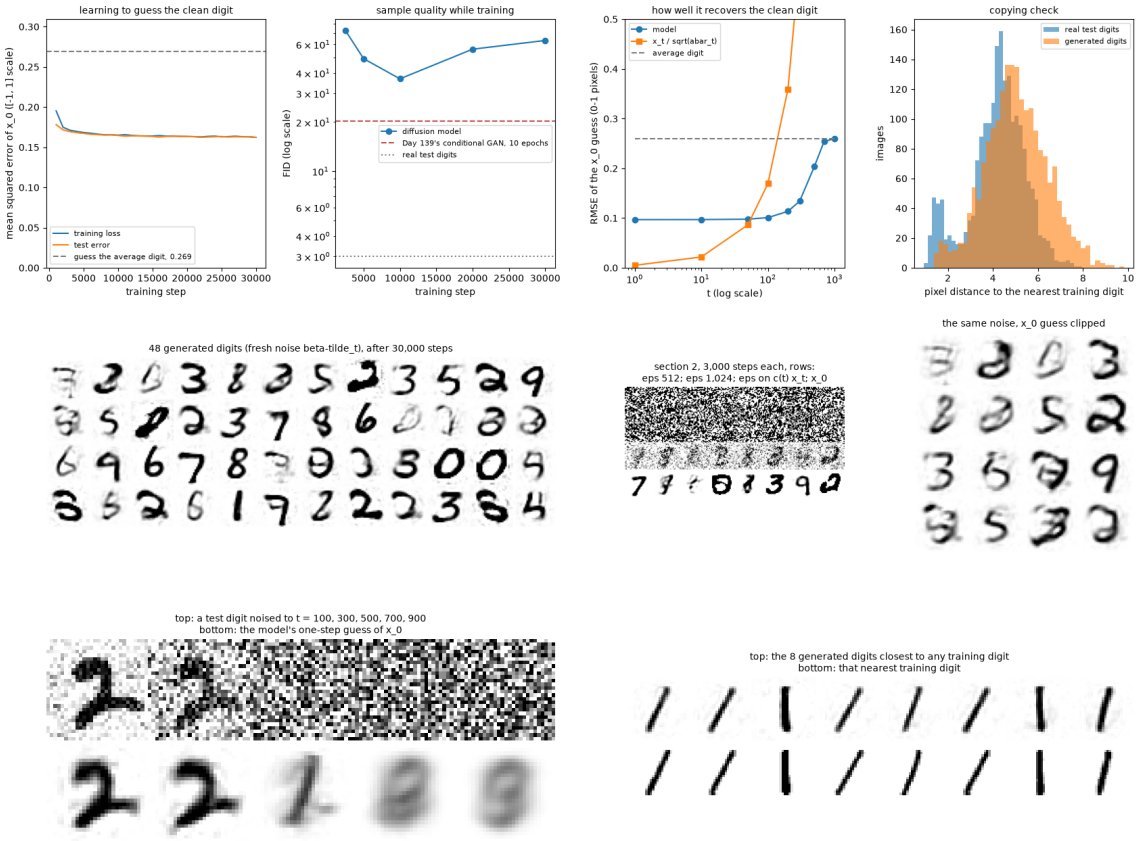
ACTUAL OUTPUT

```
(b) with the zero start, 3,000 steps each, then 2,000 samples each from the same noise, scored
('ink' is the average pixel, 0-1; the real test digits' is 0.133; Day 138's scores for them: IS 9.407, FID 2.996)
       what the output is read as     IS      FID  precision  recall    ink  train
                    eps, 512 wide  1.817   325.88      0.136   0.000  0.498   105s
                  eps, 1,024 wide  1.707   341.59      0.117   0.000  0.510   294s
  eps on top of c(t) x_t, 512 wide  2.311   238.69      0.868   0.095  0.238   121s
   x_0 (the clean digit), 512 wide  6.627    49.45      0.647   0.538  0.145   115s
the eps readings leave speckled noise in their samples (too much ink); widening to 1,024 does not cure it; reading the
same network's output as x_0 does. A likely reason, not tested further here: to output eps the network must carry 784
independent noise values through 512 (or 1,024) ReLUs, while x_0 is a digit, far simpler to describe; reading the
output as x_0 hands the noise part, x_t itself, to the formula eps = (x_t - sqrt(abar) x_0) / sqrt(1 - abar) for free
Actual output from this lesson's run.
```

What it means

Reading the output as ε produced speckled noise: ink 0.498 against real digits' 0.133, recall 0.000. Doubling the width did not help (FID 341.59). Adding the straight line c(t) x_t to the output helped partly (238.69). Reading the same network's output as x_0, with nothing else changed, gave FID 49.45, recall 0.538, and ink close to real (0.145).

A likely reason, not tested further here: an ε guess is 784 independent noise values, which must all pass through the hidden layers; an x_0 guess is a digit, which has far fewer degrees of freedom, and the noise is then x_t minus it, computed exactly by the formula. Real diffusion models predict ε successfully with U-Nets, whose skip connections carry the input's detail straight to the output; this small dense network has no such path.



Top: training loss and test error; FID while training; x_0 error by noise level; the copying check. Middle: 48 samples after 30,000 steps; section 2's drafts (rows: ε 512, ε 1,024, ε on c(t) x_t, x_0); samples with the x_0 guess clipped. Bottom: one test digit noised and guessed back in one step; the 8 samples closest to a training digit.

WHY CHOOSE THIS?

Small, checkable changes (one initial value, one reading of the output) separated a network that cannot learn from one that generates digits.

WHY NOT THIS? (TRADEOFFS)

Each draft ran once, with one seed; the gaps are very large, but the explanations are hypotheses.

3. Training the model, and scoring its samples while it trains

DEFINITION — start here, no prior knowledge assumed

The model reads its output as x_0, starts its output layer at zero, and trains for 30,000 steps on the squared error of the x_0 guess. Every 1,000 steps its error is measured on 1,000 fixed test digits, each with its own t and noise. At five points 2,000 samples are drawn from the same starting noise and scored by Day 138's metrics.

REAL-WORLD SCENARIO

A cook tasting the dish, not just checking the timer: the recipe's numbers can look fine while the dish itself has gone off.

MATHEMATICS

$$loss = mean\ |x_0 - f_\theta(\sqrt{A_t}\, x_0 + \sqrt{(1 - A_t)}\, \varepsilon, t)|^2, \quad t \text{ uniform on 1–1,000}$$

$$\text{for sampling: } \varepsilon' = (x_t - \sqrt{A_t}\, f_\theta(x_t, t)) / \sqrt{(1 - A_t)}, \text{ then Day 140's step}$$

IMPLEMENTATION

```
def train_image_denoiser(X2, steps, kind="x0", C=None, sizes=SIZES, zero_out=True, seed=0,
    lr=1e-3, batch=128,
                         every=1000, Xev=None, hook=None):
    net = d126.init_net(sizes, seed); opt = d126.Adam(net, lr)
    if zero_out:
        net[-1] = (net[-1][0] * 0, net[-1][1])
    rng = np.random.RandomState(seed); log = []; run = np.zeros(3)
    for s in range(1, steps + 1):
        x0 = X2[rng.randint(0, len(X2), batch)].astype(np.float64); t = rng.randint(1, len(BETAS)
            + 1, batch)
        e = rng.standard_normal(x0.shape); xt = np.sqrt(ABAR[t])[:, None] * x0 + np.sqrt(1 -
            ABAR[t])[:, None] * e
        A, eh, x0h = guess(net, xt, t, kind, C)
        miss = x0h - x0 if kind == "x0" else eh - e
        grads, _ = d126.backward(net, A, ACTS, (miss * 2 / miss.size).astype(np.float32))
        opt.step(net, grads)
        run += ((miss ** 2).mean(), (A[1] > 0).mean(), (A[2] > 0).mean())
        if s % every == 0:
            ev = eval_net(net, *Xev, kind=kind, C=C)[0 if kind == "x0" else 1] if Xev is not None
                else np.nan
            log.append((s, *(run / every), ev)); run[:] = 0
        if hook is not None:
            hook(s, net)
    return net, log
```

ACTUAL OUTPUT

```
816 -> 512 -> 512 -> 784 (ReLU); Adam lr 0.001; batches of 128 training digits; 30,000 steps;
loss: mean squared error of the x_0 guess (pixels on the [-1, 1] scale), t uniform on 1-1,000
test error: 1,000 test digits, each with its own fixed random t and eps; guessing the average training digit scores 0.2689
scores: 2,000 samples from the same noise (seed 1), Day 140's reverse_sample with fresh noise beta-tilde_t, scored by Day
    138's
Scorer (the real test digits: IS 9.407, FID 2.996, precision 0.855, recall 0.884, nearest 4.49)
    step  train loss  test error
   1,000      0.1953      0.1781
   2,000      0.1749      0.1716
   5,000      0.1684      0.1672
  10,000      0.1651      0.1646
  15,000      0.1640      0.1634
  20,000      0.1635      0.1634
  25,000      0.1630      0.1634
  30,000      0.1623      0.1623
    step      IS      FID  precision  recall  nearest
   2,500   6.532   72.918      0.574   0.462     4.93
   5,000   7.325   48.992      0.617   0.620     4.67
  10,000   6.916   36.856      0.739   0.513     4.57
  20,000   6.692   56.008      0.629   0.601     5.30
  30,000   6.809   63.350      0.652   0.535     5.24
training 1088 s; sampling 2,000 digits, 1,000 network calls each, 107 s a time
Actual output from this lesson's run.
```

What it means

The test error fell all the way, from 0.1781 at step 1,000 to 0.1623 at 30,000, against 0.2689 for guessing the average digit. The samples did not follow. FID was best at 10,000 steps (36.856, precision 0.739) and then rose to 56.008 and 63.350. A model chosen by its loss would have been the 30,000-step one, the worse one.

A likely reason, not tested here: with a constant learning rate the weights keep moving after the loss has levelled off, and a small change in every x_0 guess is applied 1,000 times while sampling. Real diffusion models sample from an averaged copy of the weights; Day 142 tries that. This is the same lesson as Day 59's: watch a model during training, not only at the end.

WHY CHOOSE THIS?

A plain regression with a fixed target: no adversary, no collapse during training (compare Days 134–136).

WHY NOT THIS? (TRADEOFFS)

Slow to sample: 2,000 digits took 107 s on average here, 1,000 network calls each, where a GAN needs one.

4. What the trained model knows at each noise level

DEFINITION — start here, no prior knowledge assumed

The same 1,000 test digits are noised to nine fixed levels and the model's guess of x_0 is compared with the truth, as a pixel error on the 0–1 scale. Two guesses that need no training show what the model adds: $x_t / \sqrt{A_t}$ (undo the shrinking, keep the noise) and the average training digit.

REAL-WORLD SCENARIO

Testing a reader on blurred text at increasing blur: at low blur, anyone can read it; at high blur, only someone who knows the language can guess.

ACTUAL OUTPUT

```
1,000 test digits at each t. x_0 errors are pixel RMSEs on the 0-1 scale (0.1: a typical pixel off by a tenth of
black-to-white). Two guesses that need no training: undo the shrink, x_t / sqrt(abar_t); and the average training digit
     t  x_0 RMSE, model  x_t / sqrt(abar)  average digit  eps error, model
     1          0.0969            0.0050         0.2593          375.8892
    10          0.0970            0.0218         0.2593           19.8312
    50          0.0978            0.0864         0.2593            1.2828
   100          0.1009            0.1694         0.2593            0.3550
   200          0.1136            0.3596         0.2593            0.0997
   300          0.1341            0.6169         0.2593            0.0473
   500          0.2042            1.7118         0.2593            0.0142
   700          0.2545            5.9690         0.2593            0.0018
  1000          0.2595           78.6931         0.2593            0.0000
the eps error at small t is large even where the x_0 guess is good: section 1's last column, at work
Actual output from this lesson's run.
```

What it means

At low noise the model is worse than doing nothing: at t = 1 its error is 0.0969 against 0.0050 for $x_t / \sqrt{A_t}$. It never quite reproduces a clean input; its guesses are slightly blurred. From t = 100 on it wins (0.1009 against 0.1694), and at t = 300 it is far ahead (0.1341 against 0.6169). At t = 1,000 it can only return the average digit (0.2595 against 0.2593), as Day 140's exact denoiser returned the ring's centre.

Its implied ε error is huge at t = 1 (375.8892) even though the x_0 guess is decent: section 1's factor of 100 at work. This is why the ε loss and the x_0 loss train such different networks: the ε loss puts almost all its weight on the lowest noise levels.

WHY CHOOSE THIS?

Errors by noise level show where the model helps and where a trivial guess is better.

WHY NOT THIS? (TRADEOFFS)

A pixel error rewards blur: the average digit scores well at high t while looking like no digit at all.

5. Generating digits: fresh noise per step, and clipping the guess

DEFINITION — start here, no prior knowledge assumed

Two sampling choices. The fresh noise added at each step: V_t (beta-tilde, the posterior's own variance), β_t (a little more), or none, as on Day 140. And clipping: real pixels lie in [–1, 1], so a common trick is to force the x_0 guess into that range at every step before working out ε from it.

REAL-WORLD SCENARIO

A sculptor told 'never cut beyond the block's edge': a sensible rule, until it stops them shaping the corners they needed.

IMPLEMENTATION

```
def sampler_fn(net, kind="x0", C=None, clip=False):
    def eps_fn(x, t):
        _, e, x0 = guess(net, x, np.full(len(x), t), kind, C)
        if clip:
            x0 = np.clip(x0, -1, 1); e = (x - np.sqrt(ABAR[t]) * x0) / np.sqrt(1 - ABAR[t])
        return e
    return eps_fn
```

ACTUAL OUTPUT

```
2,000 samples each, from the same starting noise (seed 1); fresh noise per step: beta-tilde_t (the posterior's), beta_t, or
    none;
'clip': the x_0 guess forced into [-1, 1] at every step before the noise guess is worked out from it
'digits': how many samples Day 138's feature network names as each digit 0-9 (the real test set has about 200 each); real
    ink 0.133
     noise  clip      IS      FID  precision  recall    ink   time   digits 0-9
 posterior    no   6.809   63.350      0.652   0.535  0.161   110s   238 97 241 307 128 330 117 287 72 183
      beta    no   6.807   63.417      0.653   0.538  0.161   106s   239 97 241 307 128 330 114 287 74 183
      none    no   1.025  560.473      1.000   0.000  0.033   109s   0 0 218 1782 0 0 0 0 0 0
 posterior   yes   5.004   95.212      0.756   0.181  0.124   125s   224 64 262 395 99 402 58 232 95 169
      beta   yes   5.006   95.229      0.755   0.182  0.124   124s   225 64 261 396 98 402 58 233 94 169
      none   yes   1.002  570.705      1.000   0.000  0.037   124s   0 0 0 2000 0 0 0 0 0 0
Day 139's conditional GAN after 10 epochs, for comparison: IS 7.580, FID 20.200, precision 0.442, recall 0.573
Actual output from this lesson's run.
```

What it means

V_t and β_t gave almost identical results (FID 63.350 and 63.417): they differ only at the smallest t. Removing the fresh noise was a disaster: FID 560.473, recall 0.000, and 1782 of the 2,000 samples named as the digit 3. As on Day 140, without fresh noise every sample follows the average direction, and here almost all of them were carried to one digit. Clipping, often recommended, made things worse here (FID 95.212, recall 0.181); the clipped samples were thinner and blurrier. Why is not tested here.

Even the best setting spreads the digits unevenly: 72 eights and 97 ones against 330 fives, where the real test set has about 200 of each. For comparison, Day 139's conditional GAN reached FID 20.2 after 10 epochs; this model's best checkpoint reached 36.856.

WHY CHOOSE THIS?

Fresh noise per step is what keeps the samples varied; its exact size mattered little here.

WHY NOT THIS? (TRADEOFFS)

Tricks that help large models (clipping) can hurt a small one: each needs testing on the model at hand.

6. Has it copied its training images?

DEFINITION — start here, no prior knowledge assumed

A generator that memorised its training set would score well and be worthless (and, for real images, a privacy problem: Day 145). For each of the 2,000 samples, the nearest of the 60,000 training digits is found by pixel distance. 2,000 real test digits, never trained on, give the distances an honest new digit lands at by chance.

REAL-WORLD SCENARIO

Checking a student's essay against every essay on file: the question is not whether it resembles one, but whether it resembles one more than an honest essay would.

IMPLEMENTATION

```
def nearest_train(F, Xtr, chunk=250):
    n2 = (Xtr.astype(np.float32) ** 2).sum(1); dist, idx = [], []
    for i in range(0, len(F), chunk):
        D = (F[i:i + chunk] ** 2).sum(1)[:, None] + n2[None] - 2 * F[i:i + chunk] @ Xtr.T
        j = D.argmin(1); idx.append(j); dist.append(np.sqrt(np.maximum(D[np.arange(len(j)), j],
            0)))
    return np.concatenate(dist), np.concatenate(idx)
```

ACTUAL OUTPUT

```
pixel distance (0-1 pixels, 784 of them) from each image to its nearest of the 60,000 training
    digits
                images    min     1%    10%  median   mean
       2,000 generated   1.37   1.71   3.29    5.00   5.04
2,000 real test digits   0.95   1.28   2.15    4.32   4.19
generated digits closer to a training digit than the closest real test digit is (0.95): 0
distinct nearest training digits among the 2,000 samples: 1399
a test digit was never trained on, so its distances show how close an honest new digit lands by
    chance
Actual output from this lesson's run.
```

What it means

No sign of copying. The closest sample is 1.37 from a training digit, while a real test digit came as close as 0.95; overall the samples sit further from the training set than real digits do (median 5.00 against 4.32). The 2,000 samples have 1399 different nearest neighbours, so they are not variations on a few favourites. In the figure the closest samples are all ones: a one is so simple that a new one lands close to an existing one by chance.

This tests copying of whole images in pixel space only, for a small model and a large training set. Day 145 trains on a tiny set, where memorisation is much more likely.

WHY CHOOSE THIS?

Cheap, direct, and calibrated by real held-out digits.

WHY NOT THIS? (TRADEOFFS)

Pixel distance misses near-copies that are shifted or slightly altered; larger studies use feature distances too.

Interview preparation — technical questions

Q: What is the training loop of a diffusion model?

A: Take a real image, a random t and random noise; form x_t in one step; regress the network's guess on the noise (or on x_0); repeat.

Q: Predict ε or x_0?

A: Equivalent on paper. Here, with a two-layer dense network, ε gave FID 325.88 and x_0 gave 49.45 after 3,000 steps.

Q: Why did the first draft stop learning?

A: The second hidden layer's ReLUs all switched off (share on: 0.00 by step 100); starting the output layer at zero prevented it.

Q: Why might you clip the x_0 guess, and did it help?

A: Real pixels lie in [–1, 1]; here clipping raised FID from 63.350 to 95.212.

Q: How do you check a generator is not copying?

A: Distances to the nearest training image, compared with held-out real images: here the closest sample was 1.37 away, a test digit 0.95.

Interview preparation — theoretical questions

Q: Why can't the training loss pick the best checkpoint?

A: It measures one-step guesses; samples chain 1,000 of them. Here test error fell to the end while FID was best at 10,000 steps.

Q: Why does the ε loss weight small t so heavily?

A: A miss d in x_0 is a miss $\sqrt{(A_t/(1–A_t))}$ d in ε: about 100 at t = 1.

Q: Why do U-Nets suit ε prediction?

A: Their skip connections carry the input's full detail to the output; a narrow dense network has to squeeze the noise through its hidden layers.

Q: What does the fresh noise in each reverse step do?

A: Keeps samples varied: without it, 1,782 of 2,000 samples here were the digit 3.

Q: How does a diffusion model compare with a GAN, from today's numbers?

A: Stable training with no adversary, but 1,000 network calls per sample, and here a worse best FID (36.856 against 20.2) than Day 139's conditional GAN.

Key takeaway and what's next

> **KEY TAKEAWAY**
> A diffusion model is trained by plain regression and samples by repeating one small step. On MNIST, with a small dense network, two details decided whether it worked: an output layer started at zero, and a network asked for the clean digit rather than the noise. The model then generated recognisable digits that do not copy the training set, but its samples peaked at 10,000 steps while its loss kept falling: judge a generator by its samples.

- He initialisation: second layer 0.00 on, loss stuck at 1.000; zero output layer: loss 0.600 after 400 steps.
- After 3,000 steps: ε FID 325.88, ε on c(t) x_t 238.69, x_0 49.45.
- 30,000 steps: test error 0.1781 to 0.1623; FID 72.918, 48.992, 36.856, 56.008, 63.350.
- No fresh noise: 1,782 of 2,000 samples were 3s; clipping: FID 95.212.
- Copying check: closest sample 1.37 from a training digit, closest test digit 0.95.

Suggested watch order, if starting from zero

- 1. Denoising Diffusion Probabilistic Models from Scratch using PyTorch! (Papers in 100 Lines of Code) — a compact DDPM written from scratch, with the training loop and sampler side by side.
- 2. Generative Image Diffusion from Scratch! PyTorch Deep Learning Tutorial (Luke Ditria) — a longer from-scratch build that trains on images and generates them step by step.

Day 142 preview

- **Diffusion against VAEs and GANs**: quality, training cost and sampling cost on one scorer, an averaged copy of the weights, and DDIM's faster sampling.